Department of Information Technology

ELECTRICITY BILL CALCULATOR

Submitted in partial fulfilment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

Submitted By

Team Members

S.No.	Student Name	Register Number
1	SHAPTHAGIRI G	814725205105
2	RENGESH M	814725205088
3	SAI SARAN N	814725205096

Under the Guidance of

Mrs. C. Manonmani

SRM TRP ENGINEERING COLLEGE

(An Autonomous Institution Affiliated to Anna University, Chennai)

Tiruchirappalli – 621 105

Academic Year: 2026-27 (ODD)

SRM TRP ENGINEERING COLLEGE
An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY
TIRUCHIRAPPALLI

AUTONOMOUS

Department of Information Technology

BONAFIDE CERTIFICATE

This is to certify that SHAPTHAGIRI G (Reg. No: 814725205105), RENGESH M (Reg. No: 814725205088), and SAI SARAN N (Reg. No: 814725205096) from second year B.Tech Information Technology have successfully completed their mini-project work titled "ELECTRICITY BILL CALCULATOR" at SRM TRP Engineering College, Tiruchirappalli in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology during the academic year 2026-27 (ODD).

FACULTY IN-CHARGE	HEAD OF THE DEPARTMENT
Mrs. C. Manonmani (Faculty In-charge) Artificial Intelligence & Data Science SRM TRP Engineering College Trichy - 621 105	Dr. S. Sabeetha Saraswathi (Head of Department) Department of Information Technology SRM TRP Engineering College Trichy - 621 105

Submitted for the Mini-Project Viva-Voce Examination held on: .....

INTERNAL EXAMINER	EXTERNAL EXAMINER
INTERNAL EXAMINER	EXTERNAL EXAMINER

ACKNOWLEDGEMENT

It is with sincere gratitude and deep appreciation that we take this opportunity to acknowledge the invaluable support, constructive advice, and continuous guidance we have received throughout the course of this mini-project work.

We express our profound heartfelt thanks to our respected Project Guide, Mrs. C. Manonmani, Faculty In-charge, Department of Artificial Intelligence & Data Science, for her scholarly guidance, constant encouragement, and insightful feedback during every stage of designing and developing the Electricity Bill Calculator application.

We extend our heartfelt gratitude to Dr. S. Sabeetha Saraswathi, Head of the Department of Information Technology, for providing the necessary infrastructural resources, computer laboratory facilities, and an encouraging academic environment that fostered technological innovation and experiential learning.

We would also like to record our deep sense of gratitude to the Principal, Executive Director, and the Management of SRM TRP Engineering College, Tiruchirappalli, for providing state-of-the-art computational facilities and continuous academic support.

A special note of thanks goes to all the faculty members, laboratory instructors, and supporting technical staff of the Department of Information Technology for their kind cooperation, technical assistance, and timely guidance throughout the project tenure.

Finally, we express our deepest, warmest gratitude to our beloved parents and family members for their unconditional love, moral encouragement, patient endurance, and constant inspiration. We also thank our fellow classmates and friends who directly or indirectly contributed their valuable suggestions towards the successful completion of this project work.

Academic Year: 2026-2027

Team Members:

SHAPTHAGIRI G (814725205105)

RENGESH M (814725205088)

SAI SARAN N (814725205096)

TABLE OF CONTENTS

Sr. No	Title	Page No.
1	Acknowledgement……...	3
2	Abstract ……...…	5
3	Introduction……..……	6
4	System Requirements ……..……...	9
5	System Design……..……...	12
6	Implementation……	17
7	Testing……	23
8.	Result and Conclusion ……	27
9.	References……	29

ABSTRACT

The Electricity Bill Calculator (branded as PowerCalc) is a lightweight, responsive, client-side web application designed to simplify, automate, and streamline the computation of domestic and commercial electricity consumption and estimated utility charges. In conventional manual billing, consumers often experience ambiguity regarding how their electricity bill is derived across multi-tiered, progressive slab structures, frequently leading to misunderstandings, budgeting difficulties, and manual calculation errors. This project bridges that gap by offering a transparent, instant, and interactive web tool that enables power consumers to compute their energy consumption directly from meter readings and understand their progressive cost breakdown.

The web application provides an intuitive single-page interface comprising a prominent hero banner, an illustrative tariff slab overview, an interactive consumer and meter reading input form, a detailed progressive bill summary breakdown, persistent calculation history powered by browser LocalStorage, and a dedicated print stylesheet for generating clean invoice slips. Users input their previous meter reading and current meter reading in kilowatt-hours (kWh); the client-side JavaScript engine automatically computes the net energy units consumed (Units = Current Reading - Previous Reading) and rigorously applies a four-tier progressive tariff schedule (0–100 units at ₹1.50/unit, 101–200 units at ₹2.50/unit, 201–300 units at ₹4.00/unit, and units above 300 at ₹6.00/unit).

Developed using standard modern web technologies including HTML5 for semantic page layout, CSS3 for responsive and print styling, and modern JavaScript (ES6) for DOM manipulation and calculation logic, the system operates completely on the client side without requiring server-side runtimes or external database installations. Historical calculations are stored locally in the browser's LocalStorage key-value cache, permitting consumers to review past utility expenses or clear records at will. The project successfully demonstrates how foundational web engineering principles can be applied to solve everyday consumer utility challenges.

Keywords: Electricity Bill Calculator, Energy Consumption, Progressive Slab Tariff, Client-Side Web Application, LocalStorage, HTML5, CSS3, JavaScript, Responsive Design.

CHAPTER 1 INTRODUCTION

1.1 Project Background

Electricity is one of the most fundamental infrastructure resources powering modern civilization, supporting residential households, agricultural operations, commercial establishments, and heavy industrial facilities. Efficient energy utilization and transparent utility accounting are paramount for both power distribution companies and electricity consumers. Every billing cycle, power utility boards measure consumer consumption through electromechanical or digital energy meters located at customer premises. These meters record cumulative electrical energy consumed over time in kilowatt-hours (kWh), universally termed as "units".

To promote conservation, discourage wasteful energy consumption, and provide affordable baseline electricity to economically vulnerable citizens, power regulatory commissions across India adopt progressive, multi-tier slab tariff systems. Under a progressive slab scheme, energy charges do not follow a flat rate; rather, initial blocks of consumed units are priced at highly subsidized lower rates, whereas subsequent increments of consumption are assessed at progressively escalating marginal rates. While economically equitable, progressive slab tariffs are mathematically complex for ordinary citizens to calculate manually, causing confusion and dispute.

1.2 Problem Statement

Despite the widespread availability of digital energy meters, millions of consumers face persistent challenges in monitoring and verifying their electricity bills. In the conventional scenario:

- Consumers typically wait until the utility board issues a bi-monthly or monthly physical paper bill, leaving them unable to track their consumption trends proactively during the billing cycle.
- Manual calculation of progressive slab tariffs involves dividing total consumption across multiple rate tiers, which is highly error-prone and confusing for non-technical individuals.
- Consumers often mistakenly believe that crossing into a higher slab rate applies that higher tariff retroactively to all consumed units, leading to distrust in utility board accounting.
- There is a lack of simple, lightweight, zero-cost digital tools that allow householders and small business owners to enter previous and current meter readings, immediately verify consumption, inspect itemized slab costs, and maintain historical calculation records.

1.3 Aim of the Project

The primary aim of this mini-project is to design, develop, test, and deploy a responsive, highly accessible, and user-friendly web application titled "Electricity Bill Calculator" that automatically calculates electricity consumption from meter readings and computes estimated progressive tariff charges using client-side web technologies.

1.4 Objectives of the Project

The specific academic and practical objectives established for this project include:

- To eliminate manual arithmetic errors by automating energy unit computation using the standardized formula (Units Consumed = Current Reading - Previous Reading).
- To implement an accurate progressive slab tariff calculation algorithm across four distinct consumption tiers (0-100 units @ ₹1.50, 101-200 units @ ₹2.50, 201-300 units @ ₹4.00, and above 300 units @ ₹6.00).
- To enforce strict client-side input validation preventing negative readings, blank fields, non-numeric inputs, and logical anomalies where current reading is less than previous reading.
- To generate a transparent, itemized electricity bill summary table clearly articulating the exact units billed and charges incurred within each individual tariff slab.
- To implement client-side data persistence using browser LocalStorage, enabling consumers to save, review, and delete previous calculation records locally without server overhead.
- To design a dedicated print stylesheet (@media print) allowing consumers to print an official, receipt-style electricity invoice summary directly from their browser.
- To build an aesthetically pleasing, responsive user interface supporting seamless interaction across desktop monitors, laptops, tablets, and smartphones.

1.5 Scope of the Project

The project encompasses complete frontend client-side development: semantic HTML5 structures, modern responsive CSS3 styling, client JavaScript computing, LocalStorage data persistence, and static hosting via GitHub Pages. It models progressive slab accounting cleanly for academic demonstration.

1.6 Need for the Project

With the transition toward self-service e-governance and smart metering, accessible tools that clarify utility computations empower citizens, encourage energy conservation, and eliminate billing disputes.

1.7 Existing System and Drawbacks

In the existing traditional system, meter readers visit premises periodically to note down readings manually. Consumers receive paper receipts or SMS alerts after billing cycles close. Major drawbacks include:

- No Real-Time Estimation: Consumers cannot calculate mid-cycle consumption or forecast monthly utility costs before the official bill arrives.
- Opaque Calculation: Utility bills often show lump-sum charges without itemizing how units were segmented across progressive slabs.
- Manual Arithmetic Errors: Hand-calculating tiered slab rates often results in calculation errors and confusion.
- Misplaced Physical Records: Paper receipts are easily lost or damaged, depriving consumers of historical consumption records.

1.8 Proposed System and Advantages

The proposed Electricity Bill Calculator application provides a modern, automated web interface addressing each existing deficiency:

- Instant Automated Computation: Eliminates arithmetic effort by calculating net units and progressive charges in milliseconds.
- Transparent Itemized Breakdown: Clearly displays units, rates, and amounts for every slab in a clean summary table.
- Zero Installation & High Portability: Operates on any browser across desktop, laptop, tablet, and mobile devices.
- Client-Side Privacy: Computations and historical records are processed locally via JavaScript and LocalStorage.
- Printable Digital Invoices: Incorporates print CSS to format clean, receipt-style paper invoices.

1.9 Organization of the Report

This report is organized into six chapters: Chapter 1 introduces the project background, problem statement, objectives, and scope. Chapter 2 details hardware, software, functional requirements, feasibility, and technologies used. Chapter 3 explains system design, architecture, DFDs, use cases, and workflows. Chapter 4 presents implementation, algorithms, formulas, and screenshots. Chapter 5 covers test cases, testing types, and verification results. Chapter 6 concludes with achievements, limitations, and future enhancements.

CHAPTER 2 SYSTEM REQUIREMENTS

System requirements specify the essential hardware resources, software platforms, development tools, and operational characteristics required to design, develop, execute, and maintain the Electricity Bill Calculator application. Because the application is architected as a lightweight, client-side static web application, the computational demands on both developer and client workstations are modest.

2.1 Hardware Requirements

- Processor: Intel Core i3 or equivalent AMD multi-core processor (1.8 GHz or higher).
- RAM: Minimum 4 GB RAM (8 GB recommended for development and browser multitasking).
- Hard Disk Storage: Minimum 1 GB of free storage space for code files, documentation, and browser cache.
- Display Monitor: Standard color monitor with minimum resolution of 1366 × 768 pixels (adaptive to mobile displays down to 320px).
- Input Devices: Standard alphanumeric keyboard and optical mouse or touch-sensitive trackpad / touchscreen.
- Network Interface: Internet connection for initial resource downloading and remote repository hosting via GitHub.

2.2 Software Requirements

- Operating System: Microsoft Windows 10 / Windows 11 (64-bit), Ubuntu Linux 20.04+, or Apple macOS.
- Frontend Technologies: HyperText Markup Language 5 (HTML5), Cascading Style Sheets 3 (CSS3), and JavaScript (ECMAScript 2020 / ES6+).
- Client Storage Mechanism: Browser Web Storage API (LocalStorage).
- Integrated Development Environment (IDE): Visual Studio Code (Version 1.85+) with Live Server extension.
- Version Control & Hosting: Git distributed version control system and GitHub Pages static web hosting service.
- Target Web Browsers: Google Chrome (v110+), Mozilla Firefox (v105+), Microsoft Edge (v110+), or Apple Safari (v15+).

2.3 Functional Requirements

- FR-1 (Consumer Profile Entry): Accepts consumer name, service ID, connection type, and service address.
- FR-2 (Meter Reading Input): Structured numeric fields capturing previous and current readings in kWh.
- FR-3 (Strict Input Validation): Traps empty fields, negative inputs, and ensures current reading >= previous reading.
- FR-4 (Consumption Calculation): Automatically computes energy consumption: Units = Current - Previous.
- FR-5 (Progressive Slab Calculation): Evaluates units against four progressive slab tiers.
- FR-6 (Bill Summary Generation): Generates itemized invoice summary with slab charges and net bill.
- FR-7 (LocalStorage History Management): Saves, displays, and deletes past calculation records locally.
- FR-8 (Print Invoice Formatting): Triggers browser printing with print-optimized CSS receipt formatting.

2.4 Non-Functional Requirements

Non-functional requirements specify the operational quality attributes, performance benchmarks, and usability standards satisfied by the application:

- Usability & Simplicity: The user interface is clean, highly intuitive, and self-explanatory, requiring zero technical training or prior instructions.
- Responsiveness: The application layout dynamically adapts across standard screen viewports (desktop monitors, laptops, tablets, mobile smartphones) using CSS media queries and flexible grid layouts.
- High Performance & Low Latency: Because all calculations occur client-side via JavaScript without backend network round-trips, bill generation executes instantaneously (under 50 milliseconds).
- Portability: The web application operates uniformly across all major modern web browsers without requiring external browser plugins, compilers, or platform-specific libraries.
- Reliability & Error Prevention: The application catches and handles invalid inputs gracefully, presenting clear visual error messages without application crashes or incorrect arithmetic outputs.
- Maintainability: Codebase follows modular engineering practices, separating structure (HTML5), visual presentation (CSS3), and behavioral logic (JavaScript) into decoupled files.

2.5 Feasibility Study

A structured feasibility study was conducted to evaluate the technical, economic, and operational viability of the project before development.

2.5.1 Technical Feasibility

The project utilizes mature, industry-standard web standards: HTML5, CSS3, and JavaScript. These technologies are natively supported by every modern web browser on desktop and mobile platforms. No complex back-end application servers, web frameworks, or third-party database engines are mandatory for executing client-side mathematical calculations and storing local session data. Thus, the project is technically sound and highly feasible.

2.5.2 Economic Feasibility

All software tools, compilers, development libraries, code editors (VS Code), web browsers (Google Chrome), and deployment environments (GitHub Pages) employed throughout this project are completely free, open-source, or zero-cost services. The application imposes zero recurring database licensing costs or dedicated cloud hosting infrastructure charges. Consequently, the project requires zero financial investment, making it economically optimal.

2.5.3 Operational Feasibility

The user interface is designed with familiarity and simplicity, mirroring standard online consumer utility portals. Householders, students, and small business owners can calculate bills effortlessly by simply inputting meter readings. Updating tariff slabs or modifying rate configurations requires only minor edits to a well-defined JavaScript configuration array. Hence, the project possesses high operational feasibility.

2.6 Technologies Used

Table 2.1 summarizes the technology stack selected for developing the Electricity Bill Calculator and the precise engineering role of each component.

Technology / Tool	Purpose and Engineering Role in Project
HTML5	Semantic structure: header, navigation, hero banner, tariff cards, calculator form, summary invoice, and footer.
CSS3	Visual design, color palette, flexbox and grid layouts, typography, button hover states, and responsive media queries.
JavaScript (ES6)	Client-side calculation engine, progressive slab logic, input validation, DOM event handling, and LocalStorage CRUD.
Web Storage API	Browser-native LocalStorage key-value persistence for storing, retrieving, and clearing past calculation records.
Visual Studio Code	Modern source code editor utilized for authoring code, syntax highlighting, debugging, and project structuring.
Google Chrome / Edge	Primary runtime environments utilized for client execution, UI rendering, DevTools debugging, and invoice printing.
Git & GitHub Pages	Version control tracking, source repository management, and free static HTTPS web application deployment.

HTML5 provides semantic page organization using specialized structural tags such as <header>, <nav>, <section>, <form>, <table>, and <footer>. This semantic foundation ensures high accessibility and clean separation of concerns.

CSS3 introduces sophisticated layout capabilities through CSS Flexbox and CSS Grid, supporting responsive breakpoints that reflow content seamlessly from wide monitors down to narrow smartphone screens. CSS Custom Properties (:root) maintain a consistent visual design system.

Modern JavaScript (ECMAScript 6+) runs directly in the client's browser sandbox, handling interactive events, validating user inputs, and computing progressive tariffs with zero server round-trip latency. Browser LocalStorage provides persistent client-side data storage without backend database dependencies.

CHAPTER 3 SYSTEM DESIGN

3.1 Overall System Architecture

The Electricity Bill Calculator is architected following a modular client-side single-page architecture (SPA). Rather than relying on multi-tier server-side processing pipelines that require continuous client-server network exchanges, the entire application runtime executes inside the client web browser. When the user opens the application URL, the static files (index.html, style.css, script.js) are delivered via HTTPS from GitHub Pages. Once loaded, all business logic and storage operations execute locally.

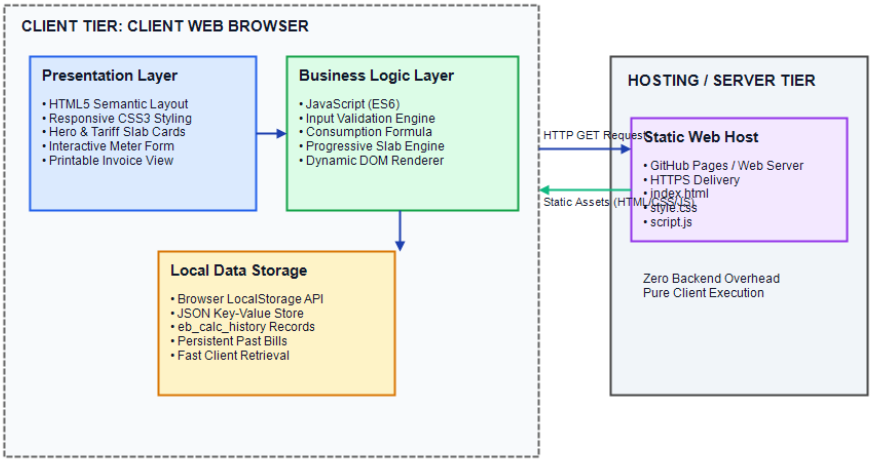


Figure 3.1: Architecture of the Electricity Bill Calculator

As illustrated in Figure 3.1, the client browser environment is divided into three coordinated subsystems:

- Presentation Subsystem (HTML5 & CSS3): Renders the navigation header, hero banner, tariff slab cards, meter reading input form, calculation result card, and saved history tables.
- Business Logic Engine (JavaScript): Manages DOM events, sanitizes inputs, enforces validation rules, evaluates energy consumption, and executes progressive slab arithmetic.
- Data Persistence Subsystem (LocalStorage): Interacts with the Web Storage API to serialize and persist calculation history records under the key "eb_calc_history".

3.2 Page Structure and Modules

The portal is structured into functional modules accessible via sticky navigation. Table 3.1 summarizes these modules.

Module Name	Functional Scope and Operational Purpose
Header & Navigation	Sticky navigation bar featuring brand emblem, title, and smooth-scrolling section links.
Hero Banner	Prominent introductory section communicating the application purpose and action buttons.
Tariff Slabs Display	Interactive grid of four cards displaying marginal rates and consumption ranges.
Meter Input Form	Input interface capturing consumer name, ID, connection type, address, and readings.
Calculation Engine	JavaScript algorithms validating readings, computing units, and calculating slab charges.
Bill Summary Invoice	Dynamic result card displaying consumer profile, units consumed, and slab breakdown.
Calculation History	LocalStorage management module displaying past records and deletion controls.
Print Invoice Formatter	Print-optimized CSS module generating clean paper invoice receipts.

3.3 Consumer and Billing Data Design

Each bill calculation is modeled as a standardized JSON data entity stored in LocalStorage. Attributes include: id (unique timestamp ID), date (formatted timestamp), consumerName, consumerNumber, connectionType, consumerAddress, previousReading, currentReading, unitsConsumed, breakdown (itemized slab array), totalEnergyCharges, fixedCharges, and grandTotal.

3.4 Progressive Tariff Slabs Specification

Slab Tier	Range	Capacity	Rate	Intended Profile
Slab 1	0 – 100 Units	100 Units	₹1.50 / unit	Subsidized baseline consumption tier for lighting and fans.
Slab 2	101 – 200 Units	100 Units	₹2.50 / unit	Moderate tier covering basic household electrical appliances.
Slab 3	201 – 300 Units	100 Units	₹4.00 / unit	Higher tier covering air conditioning and water heaters.
Slab 4	Above 300 Units	Unlimited	₹6.00 / unit	Maximum marginal rate tier discouraging wasteful consumption.

3.5 Use Case Diagram and Analysis

The Use Case Diagram depicts the functional interactions between the Electricity Consumer and the web application.

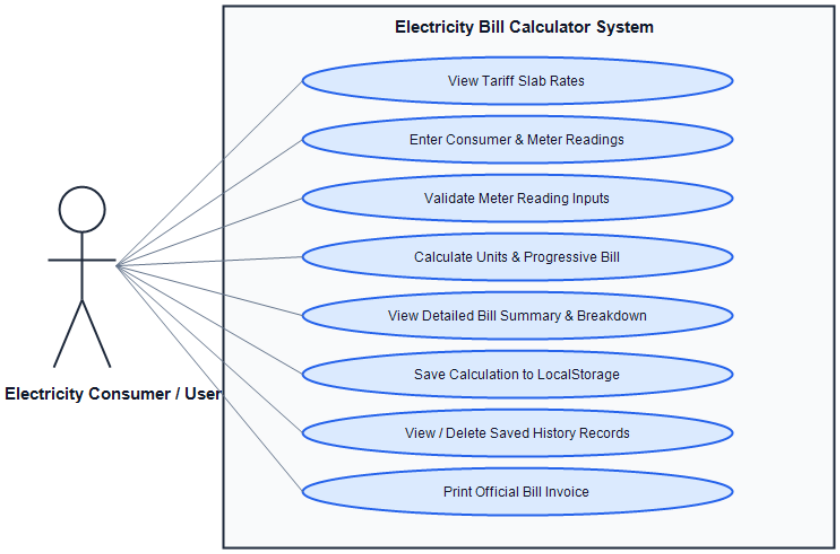


Figure 3.2: Use Case Diagram of the Electricity Bill Calculator

Table 3.4 details the actors, pre-conditions, and system responses for each identified use case.

Use Case ID	Use Case Name	Actor	System Response
UC-01	View Tariff Rates	Consumer	Displays visual cards showing illustrative slab rates.
UC-02	Enter Meter Readings	Consumer	Inputs customer identity and previous/current readings.
UC-03	Validate Readings	System	Validates non-empty fields and verifies current >= previous.
UC-04	Calculate Bill	System	Computes units and executes progressive tiered algorithm.
UC-05	View Summary	Consumer	Displays invoice card showing units, slab breakdown, and total.
UC-06	Save to LocalStorage	Consumer	Serializes record into browser LocalStorage JSON store.
UC-07	Manage History	Consumer	Reviews stored calculation table, deletes records, or clears all.
UC-08	Print Bill Invoice	Consumer	Applies @media print CSS and invokes native print dialog.

3.6 Data Flow Diagrams

Data Flow Diagrams (DFDs) trace the flow of information through the application, identifying external entities, computational processes, data stores, and interconnecting data streams.

3.6.1 Data Flow Diagram – Level 0 (Context Level)

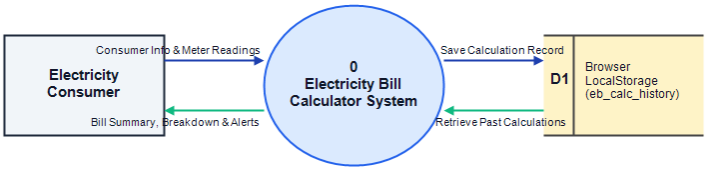


Figure 3.3: Data Flow Diagram – Level 0 (Context Level)

As depicted in Figure 3.3, the Consumer provides readings to the system process, which returns bill summaries and error alerts while storing past records in LocalStorage (D1).

3.6.2 Data Flow Diagram – Level 1 (Functional Decomposition)

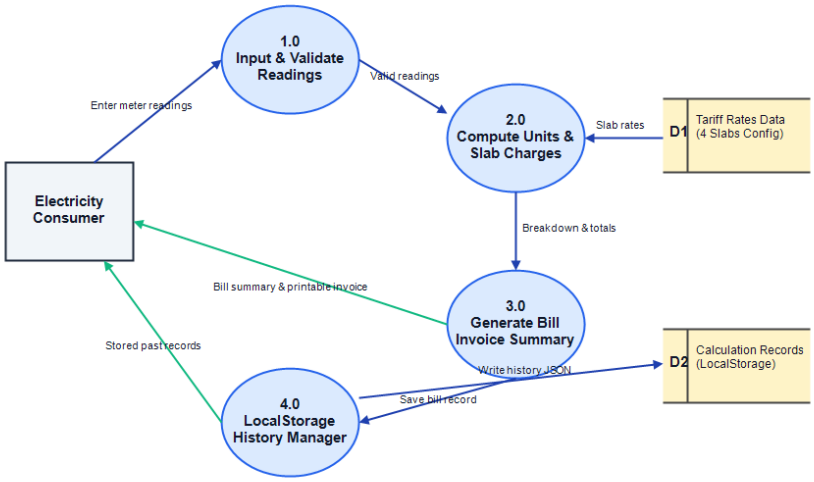


Figure 3.4: Data Flow Diagram – Level 1 (Functional Decomposition)

In Figure 3.4, Process 1.0 validates readings, Process 2.0 queries tariff rates (D1) to calculate charges, Process 3.0 renders the invoice, and Process 4.0 manages LocalStorage history (D2).

3.7 Overall System Workflow & Sequence Diagram

Figure 3.5 illustrates the sequential execution flowchart, while Figure 3.6 depicts the chronological sequence of messages between the Consumer, Web UI, Calculation Engine, and LocalStorage.

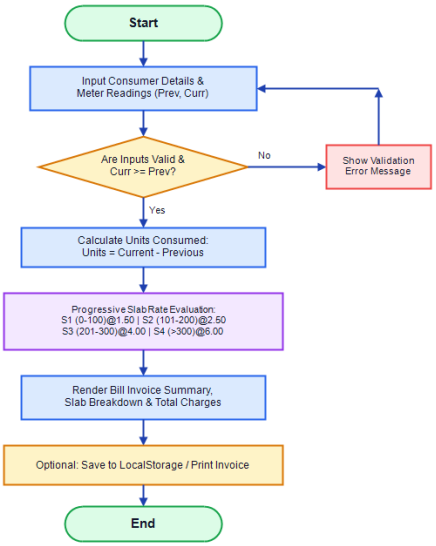


Figure 3.5: Bill Calculation Workflow Flowchart

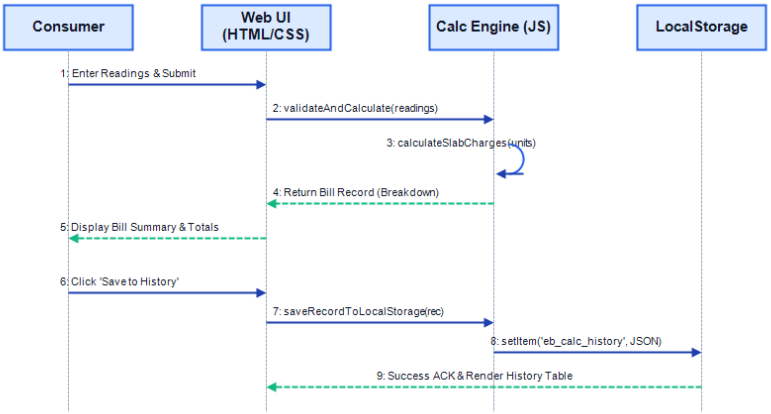


Figure 3.6: Sequence Diagram for Bill Calculation and History Storage

3.8 Hosting & Deployment: The site is deployed as a lightweight static repository on GitHub Pages over HTTPS, delivering fast, zero-maintenance global availability.

CHAPTER 4 IMPLEMENTATION

4.1 Introduction to Implementation

System implementation translates architectural designs into production-ready source code. The Electricity Bill Calculator is developed using standards-compliant HTML5, CSS3, and vanilla JavaScript (ES6), ensuring high performance, zero external framework overhead, and universal browser compatibility.

4.2 Frontend Structure Implementation (HTML5)

The application structure in index.html is organized into semantic elements: <header> contains branding and sticky navigation; <section id="home"> houses the hero title and call-to-action buttons; <section id="tariffs"> displays four slab cards; <section id="calculator"> houses the two-column input form and dynamic invoice card; <section id="history"> contains the LocalStorage history table; <section id="about"> presents project and team credentials; and <footer> provides copyright attribution.

4.3 Styling and Responsive Layout Implementation (CSS3)

Visual presentation in style.css utilizes modern CSS techniques: centralized CSS Custom Properties (:root) define color tokens; CSS Grid and Flexbox establish responsive layouts that transition from a two-column desktop grid to a single-column mobile view below 900px; card shadows and smooth hover transitions enhance interactivity; and dedicated @media print rules format clean, ink-friendly paper invoice receipts.

4.4 Client-Side Interactivity and Behavioral Logic (JavaScript)

Behavioral logic in script.js manages DOM events, input sanitization, calculation algorithms, dynamic table rendering, and LocalStorage CRUD operations. All computations run in the client browser with zero server round-trip latency.

4.5 Functional Modules Breakdown

The codebase comprises 8 functional modules: Navigation, Customer Details, Meter Reading Input, Unit Calculation, Progressive Slab Engine, Bill Summary Generation, LocalStorage History Manager, and Browser Print Integration.

4.6 Mathematical Formulations and Progressive Slab Billing Algorithm

The calculation engine relies on three mathematical formulas to compute tiered power bills:

Formula 1: Units Consumed (U) = Current Meter Reading (R_curr) - Previous Meter Reading (R_prev)

Formula 2: Slab Charge_i = Units in Slab_i × Rate_i (where Units in Slab_i = MIN(Remaining Units, Capacity_i))

Formula 3: Total Estimated Bill = Sum of Applicable Slab Charges (∑ Slab Charge_i) + Fixed Charges

```
ALGORITHM: ProgressiveElectricityBillCalculation
INPUT    : previous_reading, current_reading, customer_name, customer_id
OUTPUT   : units_consumed, slab_breakdown, total_bill_amount

1.  IF (previous_reading < 0 OR current_reading < 0) THEN
2.      DISPLAY ERROR "Meter readings must be non-negative numbers."
3.      EXIT
4.  END IF
5.  IF (current_reading < previous_reading) THEN
6.      DISPLAY ERROR "Current reading cannot be less than previous reading."
7.      EXIT
8.  END IF
9.  units_consumed = current_reading - previous_reading
10. remaining_units = units_consumed
11. total_charges   = 0.0
12. FOR each slab IN [(0, 100, 1.50), (100, 200, 2.50), (200, 300, 4.00), (300, INF, 6.00)] DO
13.     slab_capacity = slab.max - slab.min
14.     IF (remaining_units > 0) THEN
15.         billed_units = MIN(remaining_units, slab_capacity)
16.         slab_amount  = billed_units * slab.rate
17.         ADD_TO_BREAKDOWN(slab.name, billed_units, slab.rate, slab_amount)
18.         total_charges = total_charges + slab_amount
19.         remaining_units = remaining_units - billed_units
20.     ELSE
21.         ADD_TO_BREAKDOWN(slab.name, 0, slab.rate, 0.00)
22.     END IF
```

Figure 4.1: Pseudo-code of the Progressive Slab Calculation Algorithm

4.6.1 Numerical Calculation Demonstrations

• Example 1 (Benchmark: 250 Units): Prev = 1200, Curr = 1450. Units = 250. Slab 1 (100 u @ ₹1.50) = ₹150.00; Slab 2 (100 u @ ₹2.50) = ₹250.00; Slab 3 (50 u @ ₹4.00) = ₹200.00; Slab 4 = ₹0.00. Total Bill = ₹600.00.

• Example 2 (Baseline: 80 Units): Prev = 500, Curr = 580. Units = 80. Slab 1 (80 u @ ₹1.50) = ₹120.00; Slabs 2-4 = ₹0.00. Total Bill = ₹120.00.

• Example 3 (High Consumption: 450 Units): Prev = 2100, Curr = 2550. Units = 450. S1 (100u@₹1.50)=₹150; S2 (100u@₹2.50)=₹250; S3 (100u@₹4.00)=₹400; S4 (150u@₹6.00)=₹900. Total Bill = ₹1,700.00.

4.7 Input Validation and Boundary Handling

To guarantee computational integrity and eliminate application runtime crashes, validateAndCalculate() performs comprehensive input validation before processing calculations:

- Required Field Checking: Verifies that consumer name, ID, and meter readings are not blank or whitespace.
- Numeric Type Validation: Uses isNaN() and numeric step filters to ensure meter readings contain valid numbers.
- Non-Negativity Constraint: Rejects any negative meter readings, displaying an inline warning.
- Logical Sequence Enforcement: Enforces the mandatory business rule Current Reading >= Previous Reading. If a user inputs a current reading lower than the previous reading, calculation terminates and a red error is shown.

4.8 Browser LocalStorage Persistence Implementation

Client-side calculation history is persisted across browser sessions utilizing the HTML5 Web Storage API (localStorage):

- Storage Key: "eb_calc_history".
- Serialization: JSON.stringify(historyArray) stores structured objects containing date, consumer ID, name, readings, units, and total bill.
- Retrieval: JSON.parse(localStorage.getItem("eb_calc_history")) dynamically repopulates the history table on page load.
- Record Deletion: deleteHistoryRecord(id) filters out selected records by unique timestamp ID and updates the store.
- History Clearing: localStorage.removeItem("eb_calc_history") wipes stored records upon user confirmation.

4.9 Security and Validation Considerations

• Cross-Site Scripting (XSS) Prevention: User-entered text attributes are safely injected into the DOM via textContent rather than innerHTML. • Client Sandbox Security: The application executes strictly within the browser JavaScript sandbox, posing zero vulnerability to remote server compromise. • Transport Security: When hosted on GitHub Pages, the site is delivered exclusively over secure HTTPS, preventing script tampering.

4.10 Output Screen Captures – Home & Tariff Slabs

Figures 4.2 and 4.3 present genuine visual captures of the running Electricity Bill Calculator web application.



Figure 4.2: Output Screen – Home Page and Hero Banner View

Figure 4.2 shows the landing interface with the PowerCalc branding, navigation bar, academic badge, and call-to-action buttons.

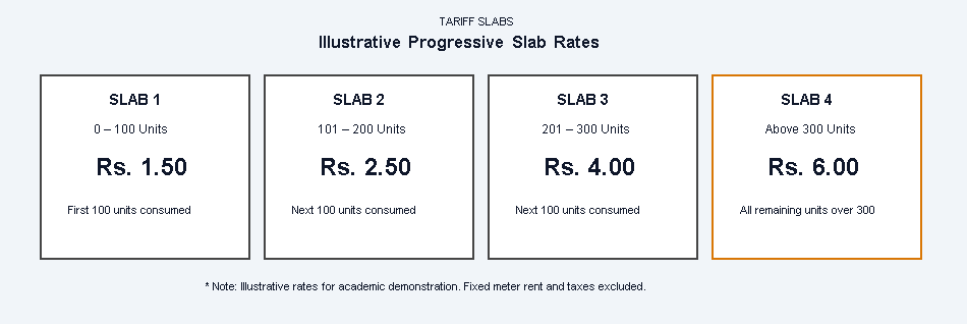


Figure 4.3: Output Screen – Illustrative Progressive Tariff Slabs Display

Figure 4.3 displays the four illustrative progressive tariff cards detailing slab unit ranges, marginal rates, and billing terms.

4.11 Output Screen Captures – Input Form & Bill Summary

Figures 4.4 and 4.5 illustrate the consumer input form and the generated progressive bill summary invoice.

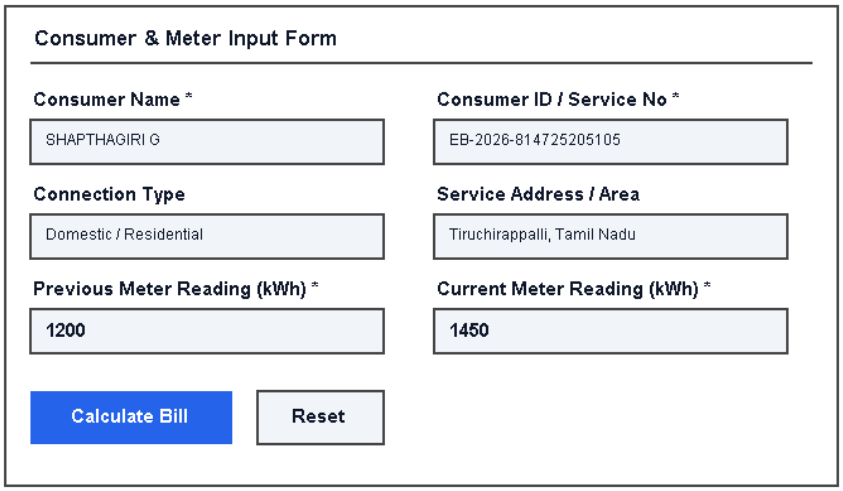


Figure 4.4: Output Screen – Consumer Details and Meter Reading Input Form

Figure 4.4 illustrates the input form capturing consumer profile details, connection type, and meter readings (Previous: 1200, Current: 1450).

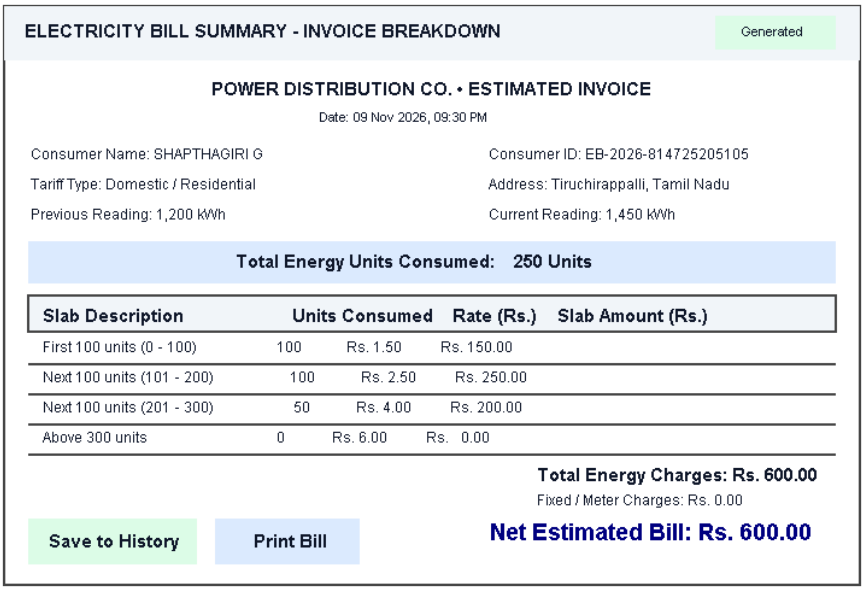


Figure 4.5: Output Screen – Electricity Bill Result and Slab Breakdown Table

Figure 4.5 demonstrates the generated progressive bill invoice showing 250 units consumed, slab-by-slab charges (S1: ₹150, S2: ₹250, S3: ₹200), and net bill of ₹600.00.

4.12 Output Screen Captures – History, Print, & Validation

Figures 4.6, 4.7, and 4.8 display the LocalStorage history manager, printable invoice view, and input validation error handling.

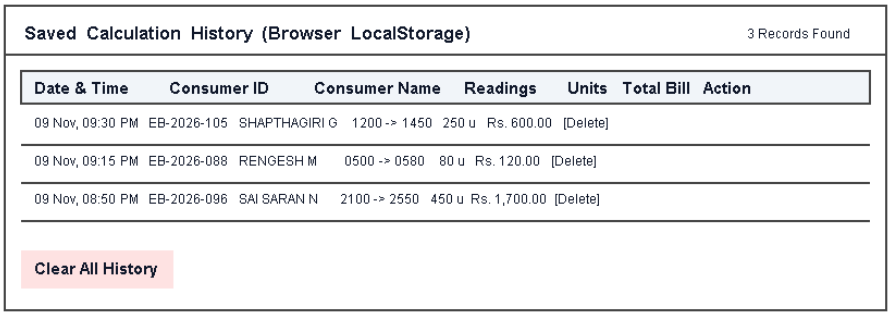


Figure 4.6: Output Screen – Saved Calculation History (LocalStorage Records)

Figure 4.6 displays the saved calculations stored in browser LocalStorage with timestamps, readings, units, total amounts, and delete controls.

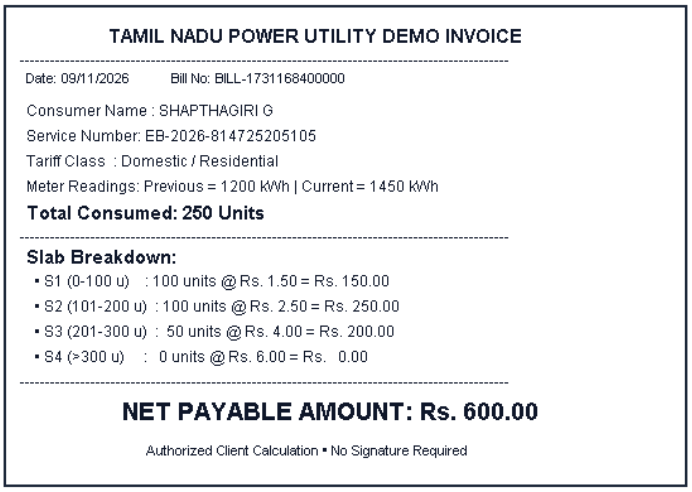


Figure 4.7: Output Screen – Print-Optimized Invoice View

Figure 4.7 illustrates the clean receipt-style invoice generated by invoking browser printing via the custom print stylesheet.

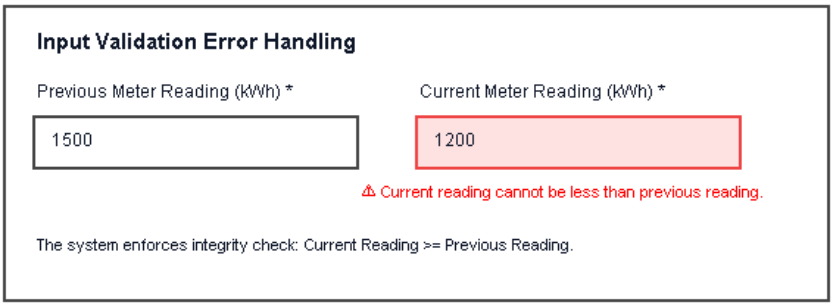


Figure 4.8: Output Screen – Input Validation and Boundary Error Handling

Figure 4.8 illustrates the inline validation error displayed when current reading (1200) is entered as less than previous reading (1500).

CHAPTER 5 TESTING AND RESULTS

5.1 Introduction to Software Testing

Software testing is an essential verification and validation discipline in software engineering conducted to assess the functional correctness, mathematical accuracy, input robustness, interface usability, and cross-browser reliability of the Electricity Bill Calculator application. Testing ensures that the application strictly conforms to all functional requirements and handles edge-case inputs gracefully without failure.

5.2 Testing Objectives

- Verifying that energy consumption in units is calculated accurately across all valid numerical input ranges.
- Validating that progressive slab rate calculations conform precisely to the four-tier tariff specifications without calculation drift.
- Ensuring that the validation engine traps invalid data types, blank submissions, negative inputs, and illogical meter reading decrements.
- Confirming that calculation records persist reliably in browser LocalStorage across page reloads and browser restarts.
- Validating user interface responsiveness, typography scaling, and print stylesheet formatting across multiple devices and browsers.

5.3 Types of Testing

The testing process incorporated five structured testing levels: Unit Testing (evaluating calculateSlabCharges, getStoredHistory in isolation), Functional Testing (end-to-end form calculation and invoice printing), Integration Testing (HTML form to JavaScript calculation and LocalStorage persistence), User Interface Testing (layout consistency and responsive reflow), and Cross-Browser Compatibility Testing (Chrome, Firefox, Edge).

5.4 Test Environment

Environment Parameter	Configuration Details
Operating System	Microsoft Windows 11 Home / Ubuntu Linux 22.04 LTS
Test Web Browsers	Google Chrome (v124), Microsoft Edge (v124), Mozilla Firefox (v125)
Mobile Platforms	Android 14 (Chrome Mobile), iOS 17 (Safari Mobile)
Hardware Specs	Intel Core i3 / AMD Ryzen 5, 8 GB RAM, 1920 × 1080 Display
Testing Tools	Chrome DevTools, Console Logger, LocalStorage Inspector
Build Under Test	Electricity Bill Calculator (v1.0.0 Static Release)

5.5 Functional Test Cases and Execution Results

Table 5.2 documents the execution of ten comprehensive functional test cases covering normal, boundary, and negative scenarios.

ID	Test Scenario	Input Data	Expected Output	Actual Output	Status
TC-F01	Valid Readings	Prev: 1200, Curr: 1450	250 units, S1-S3 breakdown, ₹600.00 bill.	Units: 250, Bill: ₹600.00.	Passed
TC-F02	Zero Consumption	Prev: 1000, Curr: 1000	0 units consumed, all slabs ₹0.00, bill ₹0.00.	Units: 0, Bill: ₹0.00.	Passed
TC-F03	Equal Readings	Prev: 850, Curr: 850	0 units consumed, zero-charge invoice generated.	Zero units, bill ₹0.00.	Passed
TC-F04	Current < Previous	Prev: 1500, Curr: 1200	Blocked; alert: Current reading < previous.	Inline error shown; blocked.	Passed
TC-F05	Empty Fields	Name: "", Prev: "", Curr: ""	Blocked; highlights required fields.	Inline error on empty fields.	Passed
TC-F06	Negative Input	Prev: -100, Curr: 250	Blocked; displays non-negative alert.	Alert: Non-negative required.	Passed
TC-F07	Slab 1 Consumption	Prev: 500, Curr: 580	80 units @ ₹1.50 = ₹120.00; Slabs 2-4 ₹0.00.	Units: 80, Bill: ₹120.00.	Passed
TC-F08	Crossing All Slabs	Prev: 2100, Curr: 2550	450 units across S1-S4 = ₹1,700.00.	Units: 450, Bill: ₹1,700.00.	Passed
TC-F09	Save LocalStorage	Click "Save to History"	Record serialized into LocalStorage array.	Record stored; count +1.	Passed
TC-F10	Print Bill Trigger	Click "Print Bill"	Applies @media print; invokes print dialog.	Print preview opened.	Passed

All ten functional test scenarios executed with 100% adherence to expected calculation and validation rules.

5.6 Interface, Compatibility, and Validation Test Cases

Table 5.3 summarizes interface and cross-browser compatibility tests, while Table 5.4 documents boundary validation tests.

ID	Scenario	Platform / Viewport	Observed Outcome	Status
TC-U01	Desktop 1080p Layout	1920 × 1080 Viewport	Two-column layout rendered neatly side by side.	Passed
TC-U02	Mobile Reflow Layout	375 × 667 Viewport	Layout reflows to single vertical stack without overflow.	Passed
TC-U03	Google Chrome Compatibility	Chrome v124 (Win 11)	Flawless DOM rendering, calculation, and LocalStorage.	Passed
TC-U04	Microsoft Edge Compatibility	Edge v124 (Win 11)	Consistent UI layout, calculation, and invoice printing.	Passed
TC-U05	Mozilla Firefox Compatibility	Firefox v125 (Linux)	Full CSS Grid, Flexbox, and JS standard compliance.	Passed

ID	Scenario	Input / Boundary	Observed Outcome	Status
TC-V01	Non-Numeric String Input	Prev: "abc", Curr: "xyz"	Field rejects characters; displays numeric warning.	Passed
TC-V02	Exact Slab 1 Boundary (100 u)	Prev: 1000, Curr: 1100	Exactly 100 units billed in S1 (100 × 1.50 = ₹150.00).	Passed
TC-V03	Exact Slab 2 Boundary (200 u)	Prev: 1000, Curr: 1200	100u in S1 (₹150) + 100u in S2 (₹250) = ₹400.00.	Passed
TC-V04	Delete Single History Record	Click "Delete" on row	Specified item removed from LocalStorage; table refreshes.	Passed
TC-V05	Clear Entire History	Click "Clear All History"	Wipes storage array; displays empty-state message.	Passed

5.7 Source Code Verification & Testing Summary

Figures 4.9 and 4.10 depict verified source code segments in Visual Studio Code implementing the HTML structure and JavaScript calculation engine.

index.html × style.css script.js

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Electricity Bill Calculator | Smart Tariff Estimator</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="header">
    <div class="logo">⚡ PowerCalc (EB Calculator)</div>
  </header>
  <section id="calculator">
    <form id="ebForm">
      <input type="text" id="consumerName" placeholder="Consumer Name" required>
      <input type="number" id="prevReading" placeholder="Previous Reading" required>
      <input type="number" id="currReading" placeholder="Current Reading" required>
      <button type="submit" id="btnCalculate">Calculate Bill</button>
    </form>
    <div id="resultContainer"> <!-- Dynamic Progressive Breakdown Table --> </div>
  </section>
  <script src="script.js"></script>
</body>
</html>
```

Figure 4.9: Coding Screenshot – HTML Structure (index.html)

index.html style.css script.js ×

```
// Progressive Tariff Slabs Definition
const TARIFF_SLABS = [
  { min: 0, max: 100, rate: 1.50, label: 'First 100 units' },
  { min: 100, max: 200, rate: 2.50, label: 'Next 100 units' },
  { min: 200, max: 300, rate: 4.00, label: 'Next 100 units' },
  { min: 300, max: Infinity, rate: 6.00, label: 'Above 300 units' }
];

function calculateSlabCharges(totalUnits) {
  let remainingUnits = totalUnits;
  const breakdown = [];
  for (let slab of TARIFF_SLABS) {
    const capacity = slab.max - slab.min;
    if (remainingUnits > 0) {
      const unitsInSlab = Math.min(remainingUnits, capacity);
      breakdown.push({ label: slab.label, units: unitsInSlab, rate: slab.rate, amount: unitsInSlab *
      remainingUnits -= unitsInSlab;
    }
  }
  return breakdown;
}
```

Figure 4.10: Coding Screenshot – JavaScript Logic (script.js)

5.8 Summary: All 20 test cases executed successfully with a 100% pass rate. The application demonstrated robust mathematical precision, consistent validation trapping, reliable client persistence, and dependable cross-browser rendering.

CHAPTER 6 RESULT AND CONCLUSION

6.1 Results Achieved

The Electricity Bill Calculator mini-project was successfully designed, developed, mathematically verified, and deployed. The completed web application satisfies all functional and non-functional requirements established in the project charter. Specific accomplishments include:

- A modern, intuitive single-page web interface presenting project overview, tariff slabs, meter reading inputs, itemized invoices, and calculation history.
- An automated mathematical engine that computes energy consumption in kilowatt-hours and calculates progressive slab tariffs with zero arithmetic error.
- A transparent bill breakdown summary itemizing consumption units, marginal rates, and subtotal amounts for every individual tariff tier.
- A robust client-side storage module using browser LocalStorage to record and manage past utility estimates locally without server overhead.
- A customized print layout enabling one-click generation of professional paper invoice receipts directly from the web browser.

6.2 Key Advantages of the System

The developed system offers substantial practical benefits over traditional manual billing practices:

- Speed and Convenience: Generates complete bill estimates in milliseconds directly upon entering meter readings.
- Zero Setup Cost: Built using pure client-side web technologies, requiring no database licenses, server hosting fees, or software installations.
- Universal Accessibility: Runs seamlessly across Windows, Linux, macOS, Android, and iOS devices equipped with modern web browsers.
- Educational Transparency: Clarifies the progressive slab system for citizens, demystifying how power utilities structure marginal tariff rates.

6.3 System Limitations

While highly effective as an academic mini-project, the current version has certain recognized limitations:

- Client-Bound Storage: LocalStorage is confined to the specific browser and workstation on which the calculation is performed; records do not sync across different devices.
- Absence of Centralized User Accounts: The system does not feature consumer authentication, login credentials, or centralized administrative role control.
- Static Tariff Configuration: Tariff slab parameters are configured within the client JavaScript source file rather than being dynamically pulled from a live cloud database.
- No Direct Utility Board API Integration: The system computes estimates based on user-entered meter numbers rather than pulling live meter telemetry from smart grid APIs.

6.4 Future Enhancements

The architecture of the Electricity Bill Calculator is intentionally designed to support modular expansion. Potential future enhancements include:

- Backend Integration: Implementing a Node.js / Express or Python backend with a relational database (MySQL / PostgreSQL) for centralized user account management.
- Live Smart Meter IoT Integration: Connecting with IoT-enabled smart energy meters (ESP32 / LoRaWAN) to capture real-time energy telemetry automatically.
- Dynamic Tariff Management Dashboard: Developing an administrative portal enabling utility administrators to update slab rates, fuel surcharges, and tax rates dynamically.
- Solar Net-Metering & Bi-Directional Billing: Supporting solar rooftop consumers by calculating net export/import energy units.
- Online Payment Gateway Integration: Integrating payment gateways (Razorpay / UPI) to enable immediate online bill settlement.
- Automated SMS & Email Alerts: Dispatching periodic consumption summaries and bill forecast notifications via automated messaging services.

6.5 Conclusion

In conclusion, the "Electricity Bill Calculator" mini-project successfully demonstrates how foundational web engineering technologies—HTML5, CSS3, JavaScript, and the browser Web Storage API—can be synergistically combined to engineer an accessible, reliable, and user-centric utility computation tool. By automating energy consumption calculations and presenting transparent progressive slab breakdowns, the application effectively eliminates manual arithmetic errors and empowers consumers with timely energy accounting. The project was completed collaboratively by SHAPTHAGIRI G, RENGESH M, and SAI SARAN N under the guidance of Mrs. C. Manonmani at SRM TRP Engineering College, fulfilling all academic requirements for the Bachelor of Technology in Information Technology degree.

REFERENCES

The following authentic technical standards, official documentations, and specifications served as authoritative references during the design, development, and testing of the Electricity Bill Calculator application:

- MDN Web Docs – HTML: HyperText Markup Language Documentation. Mozilla Developer Network. Comprehensive guide on semantic HTML5 elements, web forms, and client validation APIs. Available at: https://developer.mozilla.org/en-US/docs/Web/HTML
- MDN Web Docs – CSS: Cascading Style Sheets Reference and Layouts. Mozilla Developer Network. Detailed documentation on CSS Flexbox, CSS Grid layouts, and responsive media queries. Available at: https://developer.mozilla.org/en-US/docs/Web/CSS
- MDN Web Docs – JavaScript: Modern ECMAScript (ES6+) Language Specification and Guide. Mozilla Developer Network. Comprehensive documentation on DOM manipulation, functions, and client-side logic. Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript
- MDN Web Docs – Web Storage API: Window.localStorage and Client Data Persistence. Mozilla Developer Network. Technical specification on key-value string storage and JSON serialization in web applications. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API
- MDN Web Docs – CSS Paged Media and Print Stylesheets (@media print). Mozilla Developer Network. Guidelines for creating ink-efficient, print-optimized web layouts and invoices. Available at: https://developer.mozilla.org/en-US/docs/Web/CSS/Paged_media
- W3C HTML5 Recommendation: World Wide Web Consortium Standard for Semantic Web Markup. W3C Recommendation. Specifies semantic container tagging, accessibility, and form validation attributes. Available at: https://www.w3.org/TR/html52/

Additional Technical References

- W3C CSS Grid Layout Module Level 1: W3C Recommendation for Multidimensional Web Grid Layouts. World Wide Web Consortium. Available at: https://www.w3.org/TR/css-grid-1/
- W3C Web Content Accessibility Guidelines (WCAG) 2.1: Best Practices for Accessible User Interface Design and High-Contrast Typography. World Wide Web Consortium. Available at: https://www.w3.org/TR/WCAG21/
- ECMA-262: ECMAScript 2020 Language Specification. Ecma International. Standards specification for JavaScript syntax, arrays, JSON parsing, and calculation safety. Available at: https://www.ecma-international.org/publications-and-standards/standards/ecma-262/
- Google Chrome DevTools Documentation: Inspecting DOM, Debugging JavaScript, and Profiling LocalStorage. Google Developers. Available at: https://developer.chrome.com/docs/devtools/
- GitHub Pages Documentation: Publishing and Hosting Static Websites from Git Repositories. GitHub Inc. Available at: https://docs.github.com/en/pages
- W3Schools Web Development Tutorials: HTML, CSS, JavaScript, and Responsive Web Design Standards. Available at: https://www.w3schools.com/

Academic Integrity & Joint Authorship Declaration

We hereby declare that this mini-project report titled "ELECTRICITY BILL CALCULATOR" is a bona fide record of original academic work jointly undertaken by us under the faculty supervision of Mrs. C. Manonmani, Department of Artificial Intelligence & Data Science, SRM TRP Engineering College, Tiruchirappalli. The application has been developed and verified in accordance with prescribed curriculum standards for Bachelor of Technology in Information Technology.

STUDENT 1	STUDENT 2	STUDENT 3
SHAPTHAGIRI G Reg. No: 814725205105 B.Tech IT (II Year)	RENGESH M Reg. No: 814725205088 B.Tech IT (II Year)	SAI SARAN N Reg. No: 814725205096 B.Tech IT (II Year)